

OptArrow Developer Interview Evidence Report

Document Type: Developer-Derived Project Evidence

Project: OptArrow

Evidence Source: Semi-structured developer interview

Interview Date: 30 September 2026

Core Developer: Farid Zare

Supervisor: Zaffar Haider Janjua

Interviewer: Oluwatimilehin Ayomide Oladipo

Purpose: To capture developer-confirmed evidence concerning requirements practices, architectural decisions, design decisions, decision rationale, traceability, governance, and known process gaps that may not be explicitly represented in the software repository or visual software artefacts.

1. Purpose and Evidence Status

This report converts the raw developer interview into structured project evidence suitable for inclusion in the OptArrow evidence pool. The interview was conducted specifically to investigate evidence that was missing or incomplete following analysis of requirements, architecture and design processes. The resulting information is intended to contribute to process documentation and evidence-gap analysis.

The supervisor confirmed that the interview could serve as **basic evidence**, while noting that some practices may remain partially supported or entirely missing because OptArrow is a relatively small research project with processes that are not yet fully mature.

For evidence-pool purposes, statements in this report are classified as:

- **DEVELOPER_CONFIRMED:** Explicit statement about an existing project characteristic, decision or practice.
- **DEVELOPER_CONFIRMED_ABSENCE:** Explicit confirmation that a process, practice, mechanism or evidence does not currently exist or is not formalised.
- **DEVELOPER_REPORTED_HISTORY:** Developer recollection concerning previous approaches or development decisions that could potentially be corroborated from repository history.
- **SUPERVISOR_CONTEXT:** Contextual information supplied by the project supervisor concerning project maturity or process development.
- **NOT_CURRENT_EVIDENCE:** Hypothetical, proposed or recommended future practice. Such information must not be interpreted as evidence of the current OptArrow implementation.

2. Requirements Definition and Stakeholder Feedback

EVI-INT-001 --- Stakeholder Feedback is Collected Through GitHub Issues

Evidence classification: DEVELOPER_CONFIRMED

Finding: OptArrow currently uses the project's GitHub Issues mechanism as a means through

which users and other stakeholders can provide feedback.

Evidence interpretation: This supports the existence of a stakeholder-feedback collection mechanism. It does **not**, by itself, establish that submitted feedback undergoes a formal review, approval, prioritisation, requirements-management or traceability process.

Potential corroborating artefact: GitHub Issues associated with the OptArrow repository.

EVI-INT-002 --- Formal Requirements Traceability is Not Currently Established

Evidence classification: DEVELOPER_CONFIRMED_ABSENCE

Finding: The developer explicitly reported that OptArrow does not currently maintain a formal traceability mechanism connecting stakeholder feedback, agreements and subsequent decisions.

Evidence interpretation: GitHub Issues provide evidence of feedback collection, but the interview does not support a claim that OptArrow currently maintains end-to-end traceability from stakeholder need → requirement → decision → implementation.

EVI-INT-003 --- Some Technical Decisions Originated from Principal-Investigator Direction

Evidence classification: DEVELOPER_CONFIRMED

Finding: Some technical decisions were initiated through direction from the project's principal investigator rather than through a documented alternatives-analysis process.

Evidence interpretation: This provides contextual evidence concerning the origin of some project decisions. It does not establish that every OptArrow technical decision followed this mechanism.

3. Project Process Maturity

EVI-INT-004 --- Requirements and Related Engineering Processes Are Not Yet Fully Formalised

Evidence classification: SUPERVISOR_CONTEXT

Finding: The supervisor described relevant OptArrow processes as not yet fully formalised and characterised the project as a small research project whose processes remain comparatively immature.

Evidence interpretation: This contextualises missing process evidence identified elsewhere. A lack of formal traceability, governance or documented decision analysis should not automatically be interpreted as extraction failure.

4. Architecture Definition

EVI-INT-005 --- Architecture Evolved Iteratively Rather Than From a Formal Architecture Roadmap

Evidence classification: DEVELOPER_CONFIRMED

Finding: OptArrow's architecture evolved during development rather than being produced through a fully documented architecture-definition strategy established before implementation.

Evidence interpretation: Architecture exists and architectural decisions were made, but the process through which those decisions were reached was not consistently formalised and documented.

5. Architectural Decision Rationale

EVI-INT-006 --- Apache Arrow Was Selected for In-Memory Data Transfer

Evidence classification: DEVELOPER_CONFIRMED

Finding: One stated rationale for selecting Apache Arrow was its support for **in-memory data transfer**.

Evidence interpretation: This provides decision-rationale evidence that may not be recoverable from source code alone.

EVI-INT-007 --- Cross-Language Compatibility Was a Driver for Apache Arrow

Evidence classification: DEVELOPER_CONFIRMED

Finding: A second stated reason for selecting Apache Arrow was compatibility across programming languages.

Evidence interpretation: This is particularly relevant to OptArrow's cross-language architecture and should remain distinct from repository evidence demonstrating **how** Arrow is actually implemented.

EVI-INT-008 --- HiGHS Selection Was Influenced by Open-Source Availability

Evidence classification: DEVELOPER_CONFIRMED

Finding: HiGHS was selected partly because it is a free and open-source optimisation solver and was described as popular among developers and scientific users.

Evidence interpretation: This provides technology-selection rationale. Repository evidence should separately establish whether and how HiGHS is implemented.

6. Architectural Alternatives

EVI-INT-009 --- COBRA Toolbox Represents a Previous Architectural Approach

Evidence classification: DEVELOPER_REPORTED_HISTORY

Finding: The developer identified the COBRA Toolbox approach as a previous architecture relevant to the evolution of OptArrow.

Evidence interpretation: Where possible, this historical claim should be corroborated with project records, repository history or related project documentation.

EVI-INT-010 --- Remote Optimisation Was Considered Preferable to Workstation-Only Optimisation

Evidence classification: DEVELOPER_CONFIRMED

Finding: The project moved conceptually toward separating mathematical optimisation from the developer's local workstation.

Evidence interpretation: This provides architectural rationale for separation between client-side development activity and remote optimisation capability.

7. Architecture Governance

EVI-INT-011 --- Formal Architecture-Governance Process Is Not Currently Established

Evidence classification: DEVELOPER_CONFIRMED_ABSENCE

Finding: OptArrow does not currently have a well-established process-based approach governing architectural evolution and formal architectural changes.

Evidence interpretation: This constitutes direct evidence of a governance/process gap.

8. Design Alternatives and Technology Evaluation

EVI-INT-012 --- No Alternative Data-Transfer Protocol Was Formally Trialled for the Apache-Based Mechanism

Evidence classification: DEVELOPER_CONFIRMED_ABSENCE

Finding: The developer reported that the project did not trial an alternative protocol for the relevant Apache-based data-transfer/data-storage mechanism.

Evidence interpretation: The evidence does **not** support a claim that Apache Arrow was selected following a systematic experimental comparison of multiple competing data-transfer protocols.

EVI-INT-013 --- Alternative Implementation Routes Were Explored During Development

Evidence classification: DEVELOPER_REPORTED_HISTORY

Finding: Alternative approaches were explored in other areas of process/design implementation. Some features or approaches were developed, modified, abandoned or replaced when they did not work satisfactorily.

Evidence interpretation: This supports iterative experimentation but should not be expanded into detailed alternative-design claims without additional evidence.

EVI-INT-014 --- GitHub History May Contain Corroborating Evidence of Alternative Implementations

Evidence classification: DEVELOPER_REPORTED_HISTORY

Finding: The developer identified GitHub repository history/comments as a potential source for identifying features that were developed, changed, modified, deleted or replaced.

Evidence interpretation: A stronger evidence chain could be:

Developer statement → GitHub/repository retrieval → corroborating implementation history → supported engineering claim.

9. Confirmed Evidence Gaps

ID	Confirmed gap	Evidence status
----	---------------	-----------------

GAP-INT-001	Formal traceability between Developer-confirmed stakeholder feedback and subsequent requirements/decisions is not established.	
-------------	--	--

GAP-INT-002	Architectural definition did Developer-confirmed not originate from a consistently documented architecture strategy/roadmap.	
-------------	--	--

GAP-INT-003	Architectural decision-making Developer/supervisor and alternatives analysis are evidence not consistently formalised.	
-------------	--	--

GAP-INT-004	Formal architecture Developer-confirmed governance/change-management process is not currently well established.	
-------------	---	--

GAP-INT-005	Apache-based data-transfer Developer-confirmed technology was not selected following trials of alternative protocols.	
-------------	---	--

GAP-INT-006	Some historical alternative Developer-reported implementations are recalled history by the developer but require repository/GitHub corroboration.	
-------------	---	--

10. Information That Must Not Be Treated as Current Project Evidence

Hypothetical, ideal or future practices discussed during the interview should be classified as **RECOMMENDATION**, **PROPOSED_PRACTICE**, or **FUTURE_STATE** and must not enter the evidence pool as proof of current implementation.

Valid current evidence: Stakeholder feedback is gathered through GitHub Issues.

Valid gap evidence: Formal requirements traceability is not currently established.

Not valid as current evidence: OptArrow should establish a formal stakeholder-feedback review and traceability process.

11. Recommended Evidence-Pool Representation

Rather than storing this report only as free-form `auth_knowledge`, V6.4 should ideally represent individual interview findings as evidence units with explicit provenance.

Evidence ID: `EVI-INT-006`

Source Type: `DEVELOPER_INTERVIEW`

Evidence Classification: `DEVELOPER_CONFIRMED`

Topic: Architecture Decision Rationale

Statement: Apache Arrow was selected partly because it supports in-memo

Source: OptArrow Developer Interview

Interview Date: 30 September 2026

Speaker: Core Developer

Transcript Location: 12:54-13:19

Corroboration Status: `NOT_YET_CORROBORATED`

Related Process: Architecture Definition

Permitted Use: Engineering documentation; Evidence-gap assessment; Cros

12. Overall Evidence Assessment

The developer interview constitutes a **valuable complementary evidence source** for OptArrow because it contains information that cannot reliably be reconstructed from source code alone. In particular, it provides direct evidence concerning **stakeholder-feedback practices, architectural rationale, historical alternatives, project maturity, decision-making practices and confirmed process gaps**.

Its greatest contribution is not additional implementation detail. The repository remains the stronger source for determining **what has been implemented**. Instead, the interview explains **why certain decisions were made, how some practices currently operate, and which formal processes are explicitly absent**.

The sources should complement one another:

- **Repository evidence** → what is implemented.
- **Visual artefact evidence** → what is represented as intended design.
- **Developer interview evidence** → rationale, development context, current practices and confirmed absences.
- **ISO 33061 process reference** → criteria against which evidence coverage is assessed.

This gives V6.4 a stronger basis for triangulation: where multiple sources support the same engineering claim, confidence in the documentation can increase; where they conflict, the conflict should be surfaced; and where the developer explicitly confirms that a process is

absent, the system should preserve that absence rather than hallucinating a complete process to fill the documentation gap.